
Sleep-Time Compute as a Governed Memory Lifecycle

A position and review paper on delayed learning, memory destinations, capacity limits, and wake-sleep infrastructure

25 July 2026

Contents

Evidence-status legend

Executive abstract

Contributions and explicit nonclaims

1. The object of study

1.1 Why “models need sleep” is the wrong first question

1.2 The phase axis

1.3 The destination axis

1.4 A common optimization view

2. Chronological foundations

2.1 Biological and theoretical precedents, 1983–2016

2.2 Catastrophic interference, replay, and multiple timescales

2.3 Explicit machine wake–sleep before LLM agents

3. The 2022 PLOS result, DANN, and the limits of biological transfer

3.1 What the 2022 PLOS article actually showed

3.2 Adjacent 2022 computational models

3.3 DANN is a low-evidence branch

3.4 The no-free-resurrection boundary

4. The LLM and agent-memory lineage

4.1 MemGPT is not the later Letta sleeper

4.2 Mem0: useful external operators, version-dependent sleep status

4.3 Zep and Graphiti: temporal provenance without a global capacity law

4.4 Meta: sparse substrates and low-interference updates, not an audited recurring sleeper

4.5 Google: the strongest audited parametric sleep experiment

4.6 Microsoft: explicit external scheduling and several distinct adjacent branches

4.7 Direct 2026 branches: external, latent, and parametric

5. What sleep learns: data, operators, and objectives

5.1 A factorized sleep pipeline

5.2 Canonical evidence and raw replay

5.3 Coresets and latent replay

5.4 Generated replay and dream data

5.5 Distillation

5.6 Regularization, projection, isolation, and growth

5.7 Reinforcement learning

5.8 A minimum sleep-data manifest

6. Capacity is not one number

6.1 Six practical capacity dimensions

6.2 Total-state accounting

- 6.3 A finite-state lifetime boundary
- 6.4 Conditional parametric capacity is not a universal constant
- 6.5 How each method saturates
- 7. Candidate scaling laws and falsifiable mechanisms
 - 7.1 Reuse break-even
 - 7.2 Cadence
 - 7.3 Queue stability
 - 7.4 Byte-flow equilibrium
 - 7.5 Replay-work growth
 - 7.6 Capacity knees
 - 7.7 Parametric routing knee
 - 7.8 Clock-count and stage-count crossovers
 - 7.9 Internalization-completeness gap
 - 7.10 Recovery-substrate floor
- 8. Infrastructure: three execution/state planes and one snapshot/publication control plane
 - 8.1 Logical separation, not a biological hardware analogy
 - 8.2 The sleep transaction
 - 8.3 State scope
 - 8.4 Scheduler action space
 - 8.5 Data movement and physical placement
 - 8.6 Deletion and correction
 - 8.7 Minimum viable implementation
- 9. A matched lifetime benchmark
 - 9.1 Decision and unit
 - 9.2 Observable versus oracle information
 - 9.3 Controlled streams
 - 9.4 Methods and resource envelope
 - 9.5 The 297-cell confirmatory design
 - 9.6 Endpoints
 - 9.7 Failure is an outcome
 - 9.8 Statistical and reproducibility boundaries
- 10. Open questions and research hypotheses
 - 10.1 Is delayed execution itself beneficial?
 - 10.2 What should be admitted?
 - 10.3 Which destination should receive an item?
 - 10.4 When should sleep run?
 - 10.5 Are NREM and REM useful engineering distinctions?
 - 10.6 Can sleep safely change the memory harness?
 - 10.7 Is visible memory the complete state?
 - 10.8 Does externalization free usable model capacity?
 - 10.9 What is the right deletion unit for embodied memory?

10.10 Which state is fixed, user-local, cohort, or shared?

11. Governance, safety, and epistemic discipline

11.1 Promotion is a safety decision

11.2 Complete-state provenance

11.3 Independent evaluation

11.4 Right to be forgotten versus right not to be served

11.5 Claim-down rules

12. Synthesis: a promotion-and-demotion memory economy

13. Limitations

14. Conclusion

References

Evidence cutoff: 25 July 2026

Article type: evidence-bounded position paper and research agenda

Empirical status: this article reports no new benchmark result. All proposed experiments, candidate laws, and architecture claims remain unevaluated unless explicitly attributed to an audited source.

Evidence-status legend

This paper uses five labels to prevent literature summaries from being mistaken for new results.

Label	Meaning
[E] Established within scope	Supported by the audited primary source or released implementation, within the source’s stated model, data, and lifecycle boundary.
[I] Inference	A synthesis drawn from multiple audited sources. It is reasoned but was not directly tested as one claim.
[P] Proposal	A system, benchmark, metric, or decision rule proposed here for future evaluation.
[H] Hypothesis	A falsifiable empirical conjecture, with its intended rejection condition stated where possible.
[O] Open	Not answerable from the audited evidence or the proposed benchmark alone.

Company names identify author affiliations, official research records, or product ownership; they do not imply deployment. “Strict sleep-time compute” is a lifecycle classification, not a claim of biological fidelity or production readiness.

Executive abstract

The phrase *sleep-time compute* currently names several different things: reasoning over a known context before an unknown future query; asynchronous rewriting of an agent’s external memory; delayed compilation of experience into latent state, adapters, experts, or slow weights; and, more loosely, any continual-learning method described with a sleep metaphor. Treating these as one technique obscures the central design decision. A deployed learning system must decide **which wake experience to retain, when to transform it, which operator to apply, where to publish the result, and what to forget under finite state, compute, latency, and governance budgets.**

This article organizes that decision along two independent axes. The first is a **phase axis**: pre-/post-training before deployment, latency-critical wake or test-time learning, and delayed sleep-time transformation of an immutable wake snapshot. The second is a **destination axis**: exact external evidence; semantic text, vectors, or temporal graphs; latent or KV artifacts; user-local adapters or fast weights; and shared slow parameters. A method is strict sleep-time compute here only if it consumes accumulated wake experience, transforms it outside the current visible action’s causal path or against an isolated asynchronous snapshot, and publishes durable state that a later wake reuses.

The historical record is broader and older than the recent label. Complementary learning systems, pseudorehearsal, generative replay, bounded-synapse theory, parameter regularization, modular isolation, and explicit wake–sleep continual learning predate LLM agents. The often-cited 2022 PLOS work is a *PLOS Computational Biology* journal article showing that sleep-like spontaneous activity and STDP reduce catastrophic forgetting in a two-task spiking-network simulation; it is neither a conference result nor a general capacity theorem. The 2025 DANN manuscript is verifiable only at abstract/metadata level in the audited corpus and cannot support “zero forgetting.” In the LLM period, MemGPT supplies online tiered external memory; Letta’s 2025 paper supplies pre-query context compilation, while later Letta implementations supply a true foreground/sleeper split. Mem0 and Zep supply external-memory ingestion and lifecycle operators but not continual weight learning. Meta provides sparse memory substrates and low-interference finetuning, but no audited public recurring deployed sleep lifecycle. Google’s *Language Models Need Sleep* is the strongest audited parametric experimental lifecycle, combining fast-to-slow distillation, generated dreams, SFT, and RL inside a finite expert pool. Microsoft’s HIMA is the clearest explicit scheduled external-memory architecture, while several other Microsoft-linked systems occupy task-boundary, wake-time, within-query, or control-program positions rather than one uniform sleep category.

The evidence rejects two simple answers. “Store everything externally and retrieve it” moves pressure into storage growth, retrieval competition, index refresh, staleness, and deletion debt. “Compile everything into weights” moves pressure into interference, loss of plasticity, finite slot or rank, routing, model residency, correction, and unlearning. Generated replay is not memory-free: past information remains in a generator, teacher, sufficient statistic, canonical anchor, or residual

parameter trace. A finite-state system receiving a positive-entropy open-ended stream cannot retain every independent item forever at constant distortion. The practical objective is therefore a **promotion-and-demotion lifecycle**, not a one-way migration to “long-term memory.”

We propose a research program with four linked outputs. First, a total-state accounting identity counts raw, external, latent, parametric, auxiliary, lineage, candidate, migration, replica, and recovery state. Second, falsifiable candidate relations cover reuse break-even, cadence, queue stability, byte-flow equilibrium, replay-work growth, capacity knees, routing knees, and governance-dependent destination shifts. Third, a wake-snapshot-sleep-promotion-memory architecture treats consolidation as a versioned transaction with canary validation, atomic publication, rollback, and deletion fan-out. Fourth, a proposed, preregistration-ready 297-cell lifetime benchmark compares raw external, compressed external, latent/KV, user-LoRA, static-mixture, and past-only hybrid methods under common information and resource envelopes. The intended contribution is not a claim that machines biologically sleep, nor a universal scaling exponent. It is a bounded framework for making delayed learning measurable, falsifiable, and governable across memory substrates.

Contributions and explicit nonclaims

Contributions

This paper makes six bounded contributions.

1. **[I] A two-axis field map.** It separates learning phase from memory destination and applies a strict three-condition lifecycle test to the literature.
2. **[I] A chronological synthesis.** It connects biological and computational precedents, early explicit wake-sleep algorithms, LLM context compilation, deployed external-memory systems, and parametric memory research without collapsing their evidence levels.
3. **[I/P] A training and data factorization.** It decomposes sleep into admission, corpus construction, objective, destination, validation, and publication rather than treating named systems as indivisible packages.
4. **[P/H] A capacity and scaling agenda.** It supplies accounting identities, necessary bounds, restricted analytic optima, empirical candidate surfaces, and explicit falsifiers.
5. **[P] A transactional infrastructure blueprint.** It connects wake, sleep, and long-term-memory planes through immutable snapshots and generation-bound publication rather than in-place background mutation.
6. **[P] A matched lifetime benchmark.** It specifies controlled streams, future-information isolation, resource envelopes, primary endpoints, failure handling, and null-result paths.

Nonclaims

This paper does **not** claim:

- to invent wake–sleep learning, replay, dreaming, memory routing, versioned state, or background compaction;
- that NREM and REM map one-to-one onto particular machine-learning objectives;
- that the reported biological or bounded-synapse exponents transfer to transformers;
- that two or 3.6 bits per parameter is a universal natural-language memory constant;
- that any company operates the complete architecture proposed here;
- that async execution alone creates a learning benefit;
- that a compact prompt, top- k retrieval result, active cache, or expert pool implies bounded lifetime memory;
- that behavioral forgetting is physical deletion;
- that the unexecuted PAPER-C benchmark has produced a positive or negative result;
- priority beyond the bounded local primary-source audit on which this review is based.

1. The object of study

1.1 Why “models need sleep” is the wrong first question

The biological metaphor is intuitively productive because it suggests separation between fast experience and slower reorganization. It is scientifically dangerous when it substitutes for an operational boundary. A service can call a foreground function `sleep`, run an `async` method before returning, or periodically summarize a context without demonstrating a later-wake learning lifecycle. Conversely, a task-boundary continual learner may instantiate the relevant computation without using the word.

The actionable question is:

Given accumulated wake experience, future-use uncertainty, and finite lifecycle resources, which evidence should be transformed by which operator into which durable destination, at what time, with what validation and recovery contract?

This formulation makes three ideas central. First, **time of execution** and **kind of state changed** are independent. Second, “memory” includes the full state necessary to reproduce later behavior, not only the user-visible artifact. Third, capacity is a vector of storage, retention, addressability, plasticity, service, and governance constraints.

1.2 The phase axis

We distinguish three learning regimes.

Pre-/post-training before deployment

Pretraining, supervised finetuning, RL, distillation, architecture search, and memory-policy training operate on curated corpora before the evaluated deployed stream. They may create an update rule or memory substrate, but they do not themselves demonstrate learning from deployed wake experience.

Wake or test-time learning

Wake computation lies on the current request, token, task, or action path. It includes inference, retrieval, raw-event capture, immediate graph writes, online preference updates, per-token neural fast-memory updates, and test-time training. Wake can produce durable state; the defining constraint is that its update participates in or blocks the current visible action. [Test-Time Training](#), [TTT layers](#), Google’s [Titans](#), and Meta’s [PAHF](#) are important comparators, but they are not strict sleep under this definition.

Sleep-time learning

Sleep-time compute transforms accumulated experience after the current visible action, or concurrently against an immutable snapshot, and publishes a durable state for a later wake. It need not occur at night or during global idleness. In a continuously serving system, wake and sleep may overlap physically as long as the snapshot and publication boundary are explicit.

We use the following strict test:

1. **Wake-derived input:** accumulated deployed or experimental wake experience enters the transformation.
2. **Off-path boundary:** transformation occurs after the visible action or on an isolated snapshot outside its causal critical path.
3. **Later-wake persistence:** transformed state is durably published and consumed in a later wake.

This test is intentionally stricter than “offline training” and broader than “model weight update.”

1.3 The destination axis

The second axis asks where the result lives.

Destination	Representative state	Strength	Characteristic liability
Exact external	event log, transcript, document, trajectory	fidelity, audit, correction, deletion	storage and search growth
Semantic external	summary, profile, skill, vector, temporal graph	editability, compact retrieval, provenance	lossy rewrite, contradiction, index drift
Latent external	recurrent state, KV artifact, cartridge, latent prefix	short live context, fast reuse	opaque information, model/version dependence, weak portability
User/session parametric	LoRA, expert, sparse row, fast weight	repeated-use latency and local specialization	interference, routing, residency, version growth
Shared slow parametric	backbone or shared expert	maximum reuse amortization	privacy, negative transfer, unlearning and rollback blast radius
No-write/forget	tombstone, expiry, deliberate abstention	bounded state and reduced risk	irrecoverable future loss

The usual “parametric versus external” distinction remains valuable but incomplete. A multi-LoRA system must retrieve and compose modules; a temporal graph must resolve entities and time; a latent artifact may require the exact model, tokenizer, mask history, and source-conditioned KV. Destination routing is therefore at least a two-stage problem:

event → destination

destination → item/module/version and compatible composition

delivered state → answer or action

1.4 A common optimization view

Let wake events up to time t be $e_{1:t}$, persistent memory be

$$M_t = \{M_{\text{raw}}, M_{\text{text}}, M_{\text{vector}}, M_{\text{graph}}, M_{\text{KV}}, M_{\text{adapter}}, M_{\text{fast}}, M_{\text{slow}}\},$$

and a sleep policy S_ϕ operate under budget B_s and governance constraints G :

$$M_{t+1} = S_\phi (e_{1:t}, M_t; B_s, G) .$$

A later answer policy A_θ operates under a live budget:

$$a = A_\theta (q, \text{retrieve} (M_{t+1}, q); B_{\text{live}}) .$$

The design objective is not raw answer accuracy. It includes future task loss, wake latency, live tokens, sleep compute, persistent and peak bytes, data movement, interference, staleness, false memory, privacy, correction, deletion, and recovery. Unless defensible exchange rates are frozen, these should remain a constrained vector or Pareto frontier rather than an arbitrary scalar.

2. Chronological foundations

2.1 Biological and theoretical precedents, 1983–2016

[E] [Crick and Mitchison's 1983 reverse-learning proposal](#) suggested that dream sleep might weaken parasitic attractors. It remains a speculative historical hypothesis, not evidence that REM performs a universal pruning algorithm. [Buzsáki's two-stage model](#) separated rapid hippocampal encoding from later cortical processing. [Wilson and McNaughton](#), [Skaggs and McNaughton](#), and [Louie and Wilson](#) supplied classic evidence for experience-related replay during slow-wave or REM sleep. These results do not prove a literal hippocampus-to-cortex file transfer, and replay also occurs during quiet wake.

[E] [McClelland, McNaughton, and O'Reilly's complementary learning systems theory](#) is the most durable conceptual basis for the present field: a fast episodic system can acquire individual experiences while a slow distributed system learns structure through interleaving. The [2016 update by Kumaran, Hassabis, and McClelland](#) emphasizes goal-weighted replay, schema dependence, and the possibility that consistent information can sometimes enter the slow system quickly. It therefore argues against a rigid fast/slow clock.

[E] [Robins's pseudorehearsal](#) showed that random inputs labeled by an old network can reduce catastrophic forgetting without storing the original examples. The information did not disappear: it remained in the frozen teacher's function. This “no-free-replay” lesson persists in modern synthetic dreaming.

[E] The synaptic-homeostasis line, from [Tononi and Cirelli 2003](#) to [their 2014 refinement](#), motivates capacity restoration through selective renormalization rather than indefinite potentiation. The biological evidence is mixed enough that it should guide candidate operators, not license a direct mapping between sleep stage and model update.

2.2 Catastrophic interference, replay, and multiple timescales

[E] [McCloskey and Cohen](#) and [Ratcliff](#) established that sequential training can rapidly overwrite a previously learned distributed function. This is not ordinary temporal decay; the new gradient actively changes the shared solution.

[E] Bounded-synapse theory made the capacity problem explicit. [Fusi, Drew, and Abbott's cascade model](#) distributes traces over multiple plasticity timescales. [Fusi and Abbott](#) show that bounded ongoing plasticity overwrites old traces and that additional internal precision helps only under restrictive balance conditions. [Lahiri and Ganguli](#) derive finite-state memory-frontier bounds under a Markov-synapse model. [Benna and Fusi](#) show how bidirectionally coupled fast and slow variables can produce long-lived power-law-like traces. These are strong design intuitions, but their synapse count, state count, and ideal-observer signal are not transformer parameters, memory rows, or agent accuracy.

[E] Engineering methods then moved the same pressure among resources. [Elastic Weight Consolidation](#) and [Synaptic Intelligence](#) protect important directions but accumulate constraint or importance state. [Gradient Episodic Memory](#) spends exemplars and projection compute. [Deep Generative Replay](#) moves past information into a generator and old solver. [Progressive Neural Networks](#), [PackNet](#), and [Hard Attention to the Task](#) reduce interference by consuming columns, free weights, or masks. None creates unlimited capacity.

2.3 Explicit machine wake-sleep before LLM agents

The recent label should not erase the direct algorithmic lineage.

[E] [Deep Generative Dual Memory Network](#) transfers from small task-specific short-term modules into a long-term generator/classifier during downtime using generated replay. Its evidence is limited by task-batch assumptions and incomplete reproducibility, but the lifecycle predates modern LLM sleep.

[E] [FearNet](#) stores recent examples in a hippocampal component and, every ten study sessions, mixes them with Gaussian pseudo-examples to finetune a fixed mPFC network before clearing the recent store. Class statistics still grow, and router error materially reduces performance.

[E] [Progress & Compress](#) alternates a task-specialized active column with distillation plus online EWC into a fixed-size knowledge base. It is a direct task-boundary parametric consolidation precedent.

[E] [SIESTA](#) supplies perhaps the strongest large-scale pre-LLM explicit wake/sleep loop. Wake writes OPQ-compressed frozen-encoder latents and updates a running class mean; sleep performs balanced latent replay to update most upper-network parameters. The reported ImageNet-1K result is significant evidence for delayed latent replay, but its 2.02 GB store, supervised labels, frozen lower encoder, and millions of backward updates bound the claim.

[E] [Patch-Based Contrastive Learning and Memory Consolidation](#) retrains an encoder for 300 epochs during fixed sleeps, re-embeds centroids, and prunes raw patches. It demonstrates roughly thirty percent local memory reduction with a small accuracy cost in the audited vision setting, not a bounded global lifetime store.

[E] [Spens, Burgess, and Behrens](#) go further: a recurrent PPO controller learns which offline action to perform and, in selected branches, which real or generated items to replay. The toy-domain and bounded-window scope matters, but “learned sleep operator selection” is already prior art.

3. The 2022 PLOS result, DANN, and the limits of biological transfer

3.1 What the 2022 PLOS article actually showed

[E] The frequently cited paper is Golden et al., [“Sleep prevents catastrophic forgetting in spiking neural networks by forming a joint synaptic weight representation”](#), published in *PLOS Computational Biology* in November 2022. It is a journal article, not a “PLOS conference” paper. Its earliest audited precursor was [posted in 2019](#).

The custom feed-forward spiking network learned two complementary visual-foraging tasks. During wake, input-to-hidden connections used STDP and hidden-to-output connections used reward-modulated STDP. During sleep, external input was removed; the hidden layer received Poisson stimulation based on training-time average firing, and hidden-to-output learning switched

to unsupervised STDP. Sequential Task 2 training reduced Task 1 performance to approximately chance, while interleaving Task 2 with sleep restored the two-task balance to roughly 0.70/0.68 in the reported ten simulated networks.

The appropriate conclusion is narrow:

[E] In this two-task spiking-network simulation, sleep-like spontaneous activation plus local plasticity reorganized residual synaptic traces toward a joint representation and reduced catastrophic forgetting.

Four stronger readings are unsupported. The paper did not prove a general manifold theorem; its geometry was a post hoc PCA/kernel-PCA/SVM analysis. It did not test animals, humans, LLMs, long task sequences, privacy, deletion, or scheduler design. It required recoverable Task 1 traces: once extended Task 2 learning destroyed them, sleep could not resurrect the old solution. Finally, [Sleep Replay Consolidation](#) later showed that adding only 0.75% real rehearsal substantially improves the intrinsic replay method, emphasizing the value of a small canonical anchor.

3.2 Adjacent 2022 computational models

[E] [Perturbed and Adversarial Dreaming](#) gives NREM and REM different engineered objectives: occluded reconstruction during NREM and mixed/noisy adversarial examples during REM. It is a representation-learning model, not evidence for a universal biological division of labor.

[E] [Singh, Norman, and Schapiro](#) model autonomous hippocampal–neocortical interactions. Random activity and short-term depression let the model traverse attractors without an external item scheduler; NREM couples hippocampus and neocortex, while REM removes the modeled hippocampal influence. The experimenter still fixes the stage schedule, and the tasks are synthetic categories.

Together these papers make stage factorization experimentally respectable. They do not establish “NREM equals exact replay” and “REM equals creative generation” as biological fact.

3.3 DANN is a low-evidence branch

[E] The audited DANN record is Bekir Tolga Tutuncuoglu’s 2025 SSRN manuscript, [“Dream-Augmented Neural Networks: Harnessing Synthetic Sleep for Continual Learning and Zero Forgetting”](#). The public audit could verify only metadata and abstract-level claims, not a full method, architecture, datasets, baselines, seeds, uncertainty, ablations, or code. “IEEE” in the SSRN metadata does not establish an IEEE publication or affiliation.

The abstract mentions latent-only dreams, vision and NLP, domain drift, up to sixty percent forgetting reduction, and a path toward zero forgetting. [E] Those statements are evidence that the manuscript and idea exist. [I] They are not sufficient evidence for the numerical result or for zero forgetting. DANN should therefore remain a hypothesis-generating, lowest-tier item until a complete manuscript and independent reproduction are available.

3.4 The no-free-resurrection boundary

[I] Across pseudorehearsal, generative replay, PLOS sleep, ANN–SNN conversion, latent replay, and modern weight distillation, old information survives somewhere: a teacher, generator, buffer, activation statistic, canonical sample, residual synapse, checkpoint, or external archive. Let the total retained incremental state be Ω_t . If a target X is independent of new randomness U , and $I(X; \Omega_t) = 0$, no later function of (Ω_t, U) can reconstruct the specific X without new evidence. This is the data-processing boundary applied to a lifecycle, not a novel memory theorem.

The engineering consequence is conservative: **do not delete the last reversible evidence merely because average downstream behavior appears preserved.**

4. The LLM and agent-memory lineage

4.1 MemGPT is not the later Letta sleeper

[E] [MemGPT](#) introduced a virtual-context architecture with in-context working memory, recall storage, archival storage, function-call self-editing, context-pressure warnings, and recursive summaries. Its memory operations occur in the main agent’s online loop. It supplies the systems

starting point for tiered LLM memory, but the original paper does not include a separately trained or asynchronously scheduled consolidator.

[E] [Letta's 2025 Sleep-time Compute paper](#) studies a different object. Given shared context C and unknown future queries, an offline model transforms C into a more useful C' before those queries arrive. It reports accuracy and per-query-compute improvements, including approximately five-fold less test-time compute at matched accuracy in selected settings. It does not update model weights, process a deployed event stream, implement deletion or rollback, or solve lifetime capacity. Its main contribution is the allocation principle: additional pre-query computation can be amortized when context is reused.

[E] Later Letta implementations cross the strict lifecycle boundary. In the audited Letta API version, the foreground agent returns, a separate sleeper processes unhandled message pairs, and both actors share persistent memory blocks that later wake turns read. In Letta Code, a silent reflection subagent works in an isolated Git-backed MemFS worktree, edits memory and skills, commits, merges, and recompiles future context. The design provides practical provenance, conflict isolation, archive movement, and rollback without parametric training.

The chronology matters:

MemGPT 2023	online single-agent paging and editing
Letta paper 2025	pre-query context compilation
Letta API 2025	separate foreground/sleeper actors
Letta Code 2026	Git-backed reflection, memory/skill maintenance

Treating these as one unchanged system would attribute later implementation evidence to the earlier paper.

4.2 Mem0: useful external operators, version-dependent sleep status

[E] The [Mem0 paper](#), accepted at ECAI 2025, extracts candidate facts from a conversation summary plus recent messages, retrieves similar vector memories, and asks another model to choose ADD, UPDATE, DELETE, or NOOP. Its graph variant extracts and invalidates relations. Most mutation remains on or adjacent to the response path; only selected summary work is asynchronous.

[E] The audited managed Platform V3 returns a pending event and processes an asynchronous queue, but its contemporary core path is one-pass ADD-only rather than the paper's update/delete operator. The open-source synchronous library waits for extraction, comparison, and storage. The

OpenClaw auto-dream gate can merge, remove, rewrite, and add/update/delete before a later prompt, but it runs just before that response rather than as a fully off-path background service.

Mem0 therefore demonstrates that product version and invocation ownership determine the phase. It does not continually train model weights. Expiration may hide an item from search without proving physical erasure, and platform growth remains a lifecycle concern.

4.3 Zep and Graphiti: temporal provenance without a global capacity law

[E] [Graphiti/Zep](#) combines verbatim episode nodes, entity/fact edges, and community summaries. It distinguishes event time from transaction time and generally invalidates a contradicted fact rather than erasing history. Source episode IDs give unusually strong raw-to-derived lineage.

The open-source `add_episode` path performs extraction, deduplication, embedding, invalidation, and writes before returning; documentation suggests an external task queue but does not own one. Zep Cloud returns before graph construction and exposes completion events, so it satisfies the asynchronous later-read lifecycle in the audited product configuration. Managed cross-episode observations add background synthesis, but their proprietary operator lacks independent causal evaluation.

This architecture solves neither total boundedness nor causal erase. Deleting an episode does not necessarily regenerate a shared summary or restore an edge that the episode caused to be invalidated. A bounded retrieval payload is not a bounded graph.

4.4 Meta: sparse substrates and low-interference updates, not an audited recurring sleeper

[E] Meta/FAIR's public line is important for memory substrate and interference control:

- [Gradient Episodic Memory](#) protects past loss with an episodic buffer.
- [Memory Aware Synapses](#) estimates parameter importance from unlabeled activation sensitivity.
- [Product-Key Memory](#) supplies sparse lookup into very large value pools.
- [Memory Layers at Scale](#) pretrains pools as large as 64 million keys and 128 billion memory parameters.

- [Sparse Memory Finetuning](#) ranks memory rows by task specificity and updates only selected rows, substantially reducing forgetting in the reported benchmark relative to full finetuning and LoRA.

The last two are directly relevant to a parametric sleep destination, but neither demonstrates a recurring deployed wake-sleep service. Memory Layers are a static pretraining substrate. Sparse Memory Finetuning is a task-boundary/predeployment adaptation submitted to ICLR 2026 and desk-rejected in the audited public record, not an accepted conference result. Its million-row pool remains finite and lacks a slot-exhaustion, deletion, provenance, or rollback lifecycle.

[E] [PAHF](#) adds persistent per-user preference learning from clarification and correction, but writes immediately in the wake loop. [I] The audited public Meta corpus therefore supports sparse memory, stability-plasticity control, and foreground personalization—not the claim that Meta has published a general sleep-time learning stack.

4.5 Google: the strongest audited parametric sleep experiment

Google/DeepMind's broader lineage includes EWC, Memorizing Transformers, Titans, MIRAS, ATLAS, Trellis, Nested Learning, and Memory Caching. Most update neural state per token or sequence and fail the off-path criterion even when they use multiple timescales.

[E] The decisive exception is Behrouz et al., [Language Models Need Sleep](#), whose earliest audited public record was an anonymous ICLR 2026 submission. The source should not be described as accepted ICLR work on the inspected record.

Its **Knowledge Seeding** operator uses the current fast/high-frequency model as teacher and a prospective slow model plus newly unmasked low-rank expert as student. Generalized knowledge distillation updates only the new expert; the system publishes the slow state and resets old high-frequency experts. Its **Dreaming** operator generates task-conditioned candidates, introduces novelty via an irrelevant MoE expert, ranks candidates using gradient utility and exploration, performs LoRA-SFT on selected dreams, and trains the dream generator with binary downstream reward through ReSTEM.

This is the clearest audited combination of fast-to-slow parametric transfer, distillation, synthetic data, SFT, and RL. The paper reports gains on reasoning and knowledge-incorporation benchmarks and matched-target wall-time advantages over ordinary SFT. It satisfies the strict experimental lifecycle on its sequential benchmark.

The capacity and governance boundary is equally important. The evaluated implementation preallocates five low-rank MLP blocks of dimension 64. It does not test pool exhaustion, expert merge or eviction, routing saturation, user deletion, provenance, rollback, privacy, production scheduling, or multi-run uncertainty. [E] It establishes a finite experimental parametric sleep mechanism. [I] It does not establish indefinite learning or a production sleep cluster.

4.6 Microsoft: explicit external scheduling and several distinct adjacent branches

Microsoft-linked work spans several non-equivalent phases.

[E] [Human-Inspired Memory Architecture](#) is the clearest explicit external sleep pipeline. Hot events move into episodic and semantic/graph tiers through scheduled salience scoring, deduplication, promotion, retention, pruning, gist construction, decay, maturation, and reconsolidation. A six-hour default is specified but not evaluated as a recurring six-hour deployment. Its budget curves show the core trade-off: aggressive consolidation can sharply reduce recall. Four of six named mechanisms receive meaningful ablation support; the complete architecture remains partially evidenced.

[E] [PlugMem](#) transforms completed WebArena trajectories into episodic, propositional, prescriptive, and provenance-linked graph state for later tasks, making it a task-boundary experimental pass. [MemMA](#) generates five probes after a dialogue session, tests provisional memory, repairs failures, and writes back, but its paper-aligned probe-generation pipeline is absent and the study covers one LoCoMo conversation.

[E] Microsoft [MEMENTO](#) is not sleep. It trains a model before deployment to summarize and physically evict completed reasoning blocks within one ongoing generation. Its most important result for this agenda is representational: a text-identical restart loses 15.3 points on one AIME24 comparison, and passcode information can remain recoverable in downstream KV after the source block is removed. The visible summary is therefore not the complete memory state.

[E] [Retrospective Harness Optimization](#) converts completed trajectories into persistent instructions, skills, and executable tools after a costly diagnosis, proposal, replay, and self-preference process. It is an experimental control-plane sleep boundary, not a simple memory summary. A single update can require 103 agent invocations after selection, and the same model family solves, diagnoses, proposes, and ranks. This is useful evidence for learning the memory harness itself and simultaneous warning evidence about correlated judges, executable poisoning, and unbounded version growth.

The public Microsoft line is thus strongest in scheduled external architecture and task/session/control-boundary systems, not one monolithic “Microsoft sleep” method.

4.7 Direct 2026 branches: external, latent, and parametric

Several 2026 works sharpen the boundary.

- [E] [Auto-Dreamer](#) trains an offline external-memory consolidator with GRPO and later agent reward while retaining provenance-linked source trajectories. It is a direct learned external sleeper, but not a proven hard lifetime cap.
 - [E] [Do Language Models Need Sleep?](#) compiles recent context into fixed-size recurrent fast weights, clears KV, and applies offline recurrence. The state resets at sequence start, so it is not yet a cross-session lifetime memory.
 - [E] [Cartridges](#) and [Cartridges at Scale](#) distill a source into reusable latent/KV artifacts and expose compatibility, composition, storage, and offload as systems problems.
 - [E] [Can a Language Model Learn Facts Continually in Its Weights?](#) provides negative evidence. Diverse “study” packets substantially improve sequential fact retention over bare statements, but retention still plateaus and periodic sleep-like reconsolidation protects capability more than fact survival. Forgotten facts can often be recovered from external context, supporting the interpretation of weights as an addressable cache rather than canonical archive.
 - [E] [What to Keep, What to Forget](#) formalizes query-conditioned memory compaction through rate–distortion and information-bottleneck language. It directly overlaps broad claims about cross-layer compaction, reversibility, and promotion/demotion. The remaining gap is an executable, governed, cross-substrate lifecycle—not the rate–distortion framing itself.
-

5. What sleep learns: data, operators, and objectives

5.1 A factorized sleep pipeline

[I] The literature supports no universal “sleep loss.” A credible pipeline is:

wake events

- admission and immutable snapshot
- canonical/replay/dream/abstraction corpus
- transformation or training objective
- candidate destination
- retention + acquisition + governance validation
- publish, quarantine, or roll back

Named systems mix these factors in different ways. Evaluation should vary them independently where possible.

5.2 Canonical evidence and raw replay

Exact wake records—or equivalently lossless, reversible evidence—are the most direct substrate for exact re-evaluation, deterministic rebuilding, and source-level deletion fan-out. They are expensive and sometimes cannot be retained, but the audited evidence does not justify eliminating them by default.

[E] [Tiny Episodic Memories](#) shows that even one old exemplar per class can materially help in its continual-learning benchmarks. Sleep Replay Consolidation improves sharply with 0.75% real rehearsal. Facts-in-Weights shows that context can recover behaviorally inaccessible facts. RecMem’s raw “subconscious” store is both its recovery substrate and an unbounded liability. These results motivate a small, stratified canonical anchor as a baseline, not as a proved universal optimum.

5.3 Coresets and latent replay

Replay selection may use uniform reservoirs, class balance, recency-plus-history, gradient constraints, novelty, recurrence, value, or diversity. Each embeds a bias. A class-balanced store requires labels; recurrence suppresses valuable one-offs; surprise thresholds drift; value estimators can permanently remove low-frequency future utility.

Latent replay stores frozen-encoder outputs or sufficient statistics. It reduces I/O but creates **representation expiry**. If the encoder changes, the latent may become unreadable; if the encoder is frozen indefinitely, new-domain plasticity may suffer. Every latent item therefore needs an encoder and checkpoint digest plus a rebuild, migration, or retirement policy.

5.4 Generated replay and dream data

Generated replay includes:

1. pseudo-inputs labeled by an old solver;
2. distributional samples from stored statistics;
3. counterfactual questions or curricula filtered by downstream utility;
4. cross-session procedural abstractions.

It shifts rather than removes memory. A recursive loop has the error channel:

teacher error → generated target bias → student drift → next-cycle teacher error

Facts-in-Weights provides direct warning evidence: a frozen original teacher performs much better than a repeatedly self-merged teacher. **[P]** Every recursive sleeper should therefore report teacher identity, cycle depth, generated/canonical mixture, rare-event survival, and frozen-anchor ablations.

“Dream” also hides different operations: corruption, interpolation, counterfactual generation, and abstraction. PAD’s occlusions, SIESTA’s MixUp/CutMix, Google Sleep’s generated questions, and Auto-Dreamer’s procedural replacement should not be pooled as one technique.

5.5 Distillation

Distillation can compile:

- an old solver into a new solver during replay;
- a task-active column into a fixed knowledge base;
- context into a latent cartridge;
- fast/high-frequency experts into a slow expert;
- procedural scaffolds into policy weights.

It preserves only behavior covered by the teacher-input distribution. A fact may be present in a teacher but absent from generated queries, or present in weights but inaccessible through a needed composition. Distillation studies therefore need exact, paraphrase, temporal-update, reference, composition, relevance, and boundary-awareness probes—not direct recall alone.

5.6 Regularization, projection, isolation, and growth

Regularization such as EWC or SI protects directions and consumes plastic subspace. Gradient projection such as GEM spends exemplar and constraint-solve resources. Sparse rows, adapters, experts, masks, and new columns isolate memories but consume finite pools and add routing. Recycling a low-utility slot can restore plasticity only if protected information remains recoverable elsewhere.

[E] [Loss of Plasticity in Deep Continual Learning](#) shows that the ability to acquire new tasks can collapse even when old-task retention is not the first visible failure. Small ongoing resets can help in the evaluated settings, but reset without replay is not a retention mechanism. This requires three separate probes:

retention: does the system still know the old item?
acquisition: can it still learn a new item at fixed compute?
reacquisition: can canonical evidence restore what was lost?

5.7 Reinforcement learning

RL enters sleep at four levels:

- data-generation policy;
- replay-selection policy;
- consolidation-operator policy;
- retrieval or delivery policy.

Spens et al. learn offline action selection; Google Sleep trains a dream generator; Auto-Dreamer learns external replacement; TrustMem learns transition choices; AgeMem and MemCompiler learn wake-time memory action or delivery policies and are therefore important non-sleep controls.

A deployment reward is delayed and confounded by future workload. [P] It should be treated as a constrained vector of future utility, protected retention, false-memory harm, lifecycle resource cost, and governance debt. A single benchmark reward invites short-horizon reward hacking and destructive loss of rare evidence. Off-policy evaluation is credible only when logged propensities or a sufficiently faithful simulator exist.

5.8 A minimum sleep-data manifest

[P] Every sleep corpus should bind:

wake snapshot and event-range digest
tenant, privacy, consent, and source scope
admission, sampling, deduplication, and exclusion policy
canonical, latent, and generated item counts and bytes
generator, teacher, verifier, tokenizer, prompt, and decoding versions
labels and uncertainty
old/new/rare/failure/correction mixture
augmentation and counterfactual operators
per-item provenance and delete dependency
train/calibration/future-query split
poison, contradiction, and impossible-dream filters
optimizer, objective weights, cadence, and compute budget
candidate destination and rollback artifact

Generated data cannot independently validate the same claim that generated it. When teacher, generator, and judge share a model family, they are correlated evidence.

6. Capacity is not one number

6.1 Six practical capacity dimensions

The literature uses *capacity* for different objects. We distinguish:

1. **Storage capacity:** physical bits, objects, slots, ranks, experts, parameters, and recovery copies.
2. **Retention capacity:** how long old traces remain above an operational threshold under continuing writes.
3. **Addressable capacity:** how many retained items can actually be selected and used by the reader or router.
4. **Plasticity capacity:** how much fixed-budget new acquisition remains.

5. **Service capacity:** whether verification, transformation, publication, and recovery can keep up with arrivals.
6. **Governance capacity:** whether correction, consent change, deletion, provenance, and rollback meet their deadlines.

An external system may have abundant storage but poor addressability. A sparse expert pool may retain old skills but have no free slot. A fixed model may remember old probes while losing new-learning ability. A compact graph may have an unbounded delete-rebuild queue.

6.2 Total-state accounting

[P/identity] Define retained incremental answer-bearing state:

$$B_{\text{retained}} = B_r + B_e + B_l + B_p + B_a + B_v,$$

where B_r is canonical evidence; B_e , external text/vector/graph and index; B_l , all answer-reachable latent/KV state; B_p , adapter/expert/weight deltas; B_a , optimizer, importance, generator, teacher, and router state; and B_v , lineage, versions, and recovery.

Retained state is not peak state:

$$B_{\text{peak, state}}(t) = B_{\text{retained}}(t) + B_{\text{candidate}}(t) + B_{\text{pinned}}(t) \\ + B_{\text{migration}}(t) + B_{\text{workspace}}(t) + B_{\text{replica}}(t).$$

Infrastructure must additionally count every resident base-model copy and execution workspace:

$$B_{\text{peak, physical}} = \max_t \left[B_{\text{base, resident}}(t) + B_{\text{peak, state}}(t) + B_{\text{execution}}(t) \right].$$

The three terms are measured at the same maximizing timestamp t , rather than by summing their independent maxima.

These identities prevent four common accounting errors: counting only prompt state; calling archive movement “deletion”; ignoring inactive experts or optimizer state; and pricing visible summary text while retaining source-conditioned KV.

6.3 A finite-state lifetime boundary

Let Θ_0 be frozen public side information and M_n all incremental mutable state after n admitted items. For a controlled family of conditionally independent answers Y_i , queries Q_i , distortion targets D_i , and conditional rate–distortion functions:

$$\sum_{i=1}^n R_{Y_i | Q_i, \Theta_0} (D_i) \leq I(Y_{1:n}; M_n | Q_{1:n}, \Theta_0) \leq H(M_n | \Theta_0).$$

[I/necessary under assumptions] If independent positive-entropy items continue to arrive and the mutable state remains finite, constant-distortion retention cannot continue indefinitely. A system must expand, exploit real redundancy, increase distortion, restrict horizon or admission, or deliberately forget. External memory changes the location of the bound; it does not abolish it.

This is not a universal natural-language fact count. Correlation, prior knowledge, structured answers, unequal value, and lossy goals change the rate. In practice, retrieval, interference, plasticity, and governance can bind well before the ideal information limit.

6.4 Conditional parametric capacity is not a universal constant

[E] [Allen-Zhu and Li](#) report roughly two knowledge bits per parameter under controlled factual-tuple training and flexible extraction. [Morris et al.](#) report a roughly 3.5–3.6 memorized-bits-per-parameter plateau for bfloat16 GPT-style models under uniform synthetic sequences designed to remove generalization. The targets, protocols, precision, architectures, and readouts differ. Neither is a theoretical maximum or a universal deployed-memory law.

[P] The useful experiment places the same controlled opaque payload behind external rows, latent artifacts, adapters, and parameter deltas, then measures retained information, addressability, interference, correction, and deletion under a common total-state ledger.

6.5 How each method saturates

Method family	Pressure moved into	Typical saturation
Raw replay	archive, I/O, privacy	byte and service growth

Method family	Pressure moved into	Typical saturation
Latent replay	latent bytes, encoder dependency	incompatibility and representation expiry
Generated replay	generator, teacher, generation work	drift, mode loss, recursive corruption
Regularization	anchors, importance state, protected directions	plasticity loss
Sparse isolation	slots, ranks, experts, masks, catalog	exhaustion, collision, routing and residency
External rewrite	archive, index, graph, lineage, verifier	destructive compression, stale keys, delete debt
Parametric promotion	adapters/weights, checkpoints, router	interference, hard correction and unlearning
Harness search	code, skills, tools, versions, evaluation	executable poisoning and validation backlog

The answer to “will sleep learning exceed memory capacity?” is therefore yes, but the first exceeded resource is workload-dependent.

7. Candidate scaling laws and falsifiable mechanisms

The following relations are deliberately labeled. Identities and necessary conditions are not empirical scaling laws; restricted analytic optima depend on assumptions; the remaining surfaces require held-out prediction.

7.1 Reuse break-even

If compiling one item costs C_{compile} , maintenance costs C_{maint} , and each later valid use saves $\Delta C_{\text{use}} > 0$, then:

$$n_i^* = \frac{C_{\text{compile}} + C_{\text{maint}}}{\Delta C_{\text{use}}}.$$

[P/derived candidate] This is a stationary special case. With reuse intensity $\lambda_i(t)$, validity survival $S_i(t)$, and discount rate ω :

$$R_i(H) = \int_0^H \lambda_i(t) S_i(t) e^{-\omega t} dt.$$

Promotion to destination j is admissible only if valid future value exceeds compilation, serving, interference, governance, and risk costs in a common declared unit. **[H]** High reuse and low volatility should shift stable procedures toward latent or parametric forms; low reuse, high correction, or high deletion risk should shift evidence toward reversible external memory. The hypothesis fails if one destination dominates across the preregistered reuse–volatility range under matched resources.

7.2 Cadence

Let one sleep run cost C_s , and let delay by interval τ create average cost $a\tau^p$:

$$\bar{C}(\tau) = \frac{C_s}{\tau} + a\tau^p, \quad \tau^* = \left(\frac{C_s}{ap} \right)^{1/(p+1)}.$$

For linear staleness with event rate r_e , fixed run cost F , and per-event per-time harm h :

$$J(\tau) = \frac{F}{\tau} + \frac{hr_e\tau}{2}, \quad \tau^* = \sqrt{\frac{2F}{hr_e}}.$$

[P/derived candidate] These relations fail under bursts, batch economies, deadlines, repeated-compaction damage, and capacity-triggered phase changes. **[H]** A pressure-aware trigger using novelty, reuse, interference, free capacity, staleness, backlog, privacy deadlines, and resource availability will beat a fixed clock only where its prediction benefit exceeds calibration and control overhead. A consistently superior fixed cadence would reject the learned-trigger advantage in that regime.

7.3 Queue stability

For normalized sleep jobs with arrival rate λ_s , mean service time $E[S_s]$, and m effective servers:

$$\rho_s = \frac{\lambda_s E[S_s]}{m} < 1$$

is a necessary mean-load condition, not a tail guarantee. Real systems need typed resource constraints:

$$\sum_j \lambda_j E \left[w_j^{\text{gpu}} \right] < m_{\text{gpu}}, \quad \sum_j \lambda_j E \left[d_j^{\text{io}} \right] < C_{\text{io}},$$

with analogous CPU, network, RAM-byte-second, and HBM-byte-second conditions plus instantaneous stock caps.

[H] Queue-before-storage failure. Under bursty novelty, p99 sleep-job age or deletion deadline will often fail before durable storage fills. The hypothesis is rejected in a preregistered regime if storage consistently binds first after all retries, validation, publication, and recovery work are charged.

7.4 Byte-flow equilibrium

For each physical store j :

$$\frac{dB_j}{dt} = r_{\text{admit},j} \bar{b}_{\text{in},j} - r_{\text{expire},j} \bar{b}_{\text{delete},j} - r_{\text{compact},j} \bar{b}_{\text{reclaimed},j}.$$

[P/identity] A merge ratio is not reclamation unless bytes actually leave the declared boundary. A bounded hot store feeding an unbounded archive is not lifetime equilibrium. GAM's 2,048-token live buffer feeding growing graph and archive state is a direct counterexample.

7.5 Replay-work growth

If sleep after task t replays m_i retained items from every task $i \leq t$:

$$W_T = \sum_{t=1}^T \sum_{i=1}^t m_i = \sum_{i=1}^T m_i (T - i + 1).$$

For constant $m_i = m > 0$, lifetime replay work is $\Theta(T^2)$. A surprise gate that preserves a constant fraction changes the coefficient, not the exponent. A fixed recent window reduces work but can destroy retention: the audited surprise-gated research note reports recent-five-task replay worse than no replay in its proof-of-concept.

[H] Canonical-support boundary. A fixed-byte, age- and class-stratified coreset will exhibit a workload-dependent minimum support below which rare-event and recoverability performance drops faster than average accuracy. The hypothesis fails if generator-only or recent-only replay matches the anchor hybrid across frozen rare, correction, and late-query slices.

7.6 Capacity knees

[P] A capacity knee should be anchored to a preregistered utility threshold, not selected from a visually convenient curve. Candidate models should compete against constant, log-linear, power, exponential-saturation, logistic-knee, min-bottleneck, interaction, segmented, and shape-constrained alternatives on held-out scales.

The measured surface should include:

$$Q = f(P, B_r, B_e, B_l, B_p, C_s, R, H, V, \Delta, K),$$

where P is parameter state; B . are memory classes; C_s is sleep compute; R , reuse; H , heterogeneity; V , volatility; Δ , correction/deletion pressure; and K , cycle count. The outcome is a vector of retention, acquisition, addressability, calibration, interference, staleness, corruption, governance completion, latency, energy, and movement.

An “A100-regime scaling law” should require predictive success on frozen held-out scales and workload families on that exact stack. Cross-hardware wording additionally requires a prospective powered hardware holdout. Otherwise the result is a regime-specific relation or empirical curve.

7.7 Parametric routing knee

[E] [LoRA as Knowledge Memory](#) shows that memory quality depends on training-data transformation, rank, module topology, routing, and merge. Under a matched parameter budget, oracle-routed low-rank modules can beat one larger adapter, while learned routing or poor composition can erase the advantage.

[H] At fixed total adapter parameters, modular memory should dominate one adapter only while saved interference exceeds router error, merge loss, catalog-selection cost, and residency misses. The falsifier is straightforward: among routers meeting frozen calibration and serving thresholds, if simultaneous modular-minus-single upper bounds remain below the materiality margin across all cells, the modular region is absent in the tested regime.

7.8 Clock-count and stage-count crossovers

[E] TiMem’s session/day/week/month hierarchy reduces recalled context but raises construction calls by roughly 25–30% in its reported settings. [H] More clocks or memory stages should initially improve utility per active byte, then lose once call arrival, repeated abstraction, read amplification,

and queue age dominate. If each added clock remains Pareto-improving across burst, latency, cost, and distortion holdouts, the crossover is rejected.

7.9 Internalization-completeness gap

[H] Direct recall will overstate equivalence among external, latent, and parametric destinations. The largest gaps should appear in temporal replacement, cross-memory composition, implicit relevance, or boundary awareness. Stable destination rankings across acquisition, update, reference, composition, relevance, and scope would reject this hypothesis.

7.10 Recovery-substrate floor

[H] For rare or volatile evidence, removing raw or reversible state will create a nonzero correction, late-query, and rollback floor. A bounded lossy system that matches an archive-backed system on those endpoints without retaining an equivalent trace in weights, summaries, backups, or generators would reject the floor.

8. Infrastructure: three execution/state planes and one snapshot/publication control plane

8.1 Logical separation, not a biological hardware analogy

The user-facing framing—wake cluster, sleep cluster, and long-term-memory infrastructure—is directionally correct. [I] The stronger form separates three execution/state planes and one cross-cutting control plane:

1. **Wake plane:** latency-critical inference, retrieval, minimal capture, optional fast learning, and fast state.

2. **Sleep plane:** replay, dream generation, compaction, graph update, distillation, SFT/RL, adapter packaging, rebuild, and verification.
3. **Memory plane/fabric:** versioned serving across raw, text, vector, graph, latent, adapter, and weight tiers.
4. **Snapshot/publication control plane:** immutable event ranges, source handles, policy, consent and deletion epochs, candidate validation, atomic publication, and rollback across the other three planes.

These are logical responsibilities, not a prescription for four physical clusters. They may share one physical fleet or occupy separate clusters. The correct placement depends on interference, batching, data size, model residency, privacy zones, and load.

8.2 The sleep transaction

[P] Define:

$$T(X_v, M_v, B, P) \rightarrow \widehat{M}_{v+1},$$

where X_v is an immutable wake snapshot, M_v the published manifest, B a resource envelope, P a frozen policy/configuration, and $\widehat{M}_{v+1}$ only a candidate.

The candidate becomes authoritative only after:

```
verify artifact digests
verify exact parent serving head and authorization/deletion epochs
verify model/tokenizer/index compatibility
verify key-payload and cross-store completeness
run utility, retention, acquisition, poison, privacy, and deletion canaries
write a generation-bound candidate manifest
compare-and-swap the complete serving head
retain the predecessor for bounded rollback
```

Copy-on-write prevents sleep from mutating a live serving tree. An out-of-order worker that finishes successfully against an old generation is stale, not authoritative.

8.3 State scope

Memory scope and medium should be independent:

```
shared fixed base
shared slow learned state
cohort/team state
user state
session state
ephemeral query state
```

High-delete-risk personal information should not enter shared weights in the minimum viable system. A user-local adapter may cost more per user but reduce cross-user interference and rollback blast radius.

8.4 Scheduler action space

[P] The sleep controller should choose:

```
(when, tenant/cohort, evidence range, region,
operator, destination, budget, deadline)
```

Its actions include no-op, retain exact, expire, probe/repair, deduplicate, merge, abstract, rewrite, graph update, latent compile, adapter compile, parametric promotion, demotion, recycle, rebuild, and quarantine. Learned policies may propose actions, but privacy, authorization, capacity, and promotion constraints remain a non-learned governance kernel.

Local maintenance is a first-class action. A graph-wide rewrite may be unnecessary when a topic, tenant, or subgraph is affected, but local jobs must charge boundary errors, duplicate summaries, delete propagation, and index repair.

8.5 Data movement and physical placement

Four deployment options deserve measurement:

- shared accelerators with wake-priority preemption;
- time-partitioned hardware;
- separate wake and sleep fleets;
- regional/shard-local extraction with pooled heavy training.

Separate fleets reduce wake interference but duplicate base-model residency and move state over the network. Co-location reduces movement but can damage wake p99 and sleep completion. [H] A separate sleep fleet dominates only above a load/interference threshold; below it, shared or time-partitioned capacity should win.

The system should move content-addressed deltas, manifests, and small adapter artifacts instead of full histories when source data are large and reuse is high. It should move compute toward data only when model movement and lost batching do not dominate.

8.6 Deletion and correction

Deletion is a staged transaction:

```
immediate authorization deny
quarantine in-flight descendants
remove online payload and identifier mapping
invalidate/rebuild text, vector, graph, and latent views
retire/rebuild/unlearn affected adapters
expire backups and keys under a declared deadline
probe surviving explicit and latent descendants
audit residual behavior, bytes, and lineage
```

Tombstoning is not physical erasure. Source-token removal is not semantic erasure when latent descendants remain. Exact item removal from shared weights is not assumed; the system must quarantine, rebuild, apply independently validated unlearning, or disclose unresolved residuals.

Corrections should preserve validity intervals and derivation lineage instead of blindly overwriting the old assertion. This supports temporal questions and post hoc audit.

8.7 Minimum viable implementation

[P] A credible first system should remain narrow:

1. fixed shared base;
2. per-user canonical event log;
3. versioned semantic/vector memory;
4. optional user-isolated adapter as the only parametric destination;
5. fixed and pressure-triggered sleeps;
6. deterministic baseline and one past-only router;
7. immutable candidates, canaries, atomic publication, and rollback;
8. immediate deny plus tested external/adapter rebuild;
9. active, retained, and peak-state accounting;
10. trace-driven comparison of shared and separate fleets.

Shared-weight promotion and exact parametric unlearning should remain later extensions.

9. A matched lifetime benchmark

9.1 Decision and unit

The benchmark should answer: given the same wake events and future queries, should the system retain exact evidence, compress it, compile latent state, train a user-local adapter, forget, or route dynamically?

One cycle is:

```
wake capture
→ pre-sleep probes
→ immutable snapshot
→ sleep or no-sleep action
→ validation and publication
→ post-sleep probes
→ delayed next-wake probes
```

The default proposed horizon is 128 cycles, with sensitivity at 8, 32, and 512 cycles and retrieval lags of 1, 8, 32, and 128 cycles where applicable.

9.2 Observable versus oracle information

The learner may see current event payload, time, confidence, privacy scope, provenance, known supersession, past reuse, outcomes, and resource pressure. It must not see future reuse, future contradiction/deletion, poison truth, future queries, exact answers, or the clairvoyant best destination.

Generator, file, schema, and process boundaries should enforce this separation. Diagnostic probes generated from a wake snapshot are training artifacts and require their own model, prompt, input, seed, filter, acceptance, and rejection lineage. A probe cannot contain future event IDs, evaluation-query hashes, or derived oracle answers.

9.3 Controlled streams

The stream should independently vary:

- stable exact facts and opaque random payloads;
- volatile facts and preferences with validity intervals;
- procedures and skills with success and failure trajectories;
- temporal and relational graph events;
- negative feedback;
- privacy and deletion;
- poison and prompt injection;
- low-reuse salient distractors.

Future queries should separate exact acquisition keys, exact repeats, unseen paraphrases, unseen compositions, corrections, and unrelated controls. Reusing the acquisition query is cache evidence, not semantic generalization.

9.4 Methods and resource envelope

The confirmatory deployable methods are:

1. raw versioned external evidence;
2. compressed external memory;
3. latent/KV compiler;
4. user-local LoRA;

5. calibration-tuned static mixture;
6. past-only hybrid router.

A clairvoyant oracle router is diagnostic only.

All methods share the same upper bounds on live tokens, active bytes, durable bytes, total incremental peak state, physical HBM/fleet peak, sleep FLOPs, peak accelerators, information cutoff, and privacy/delete contract. Unused budget is reported rather than forcibly spent. Search, router training, teacher/judge calls, data generation, optimizer state, hidden KV, archive, index rebuild, verification, candidate workspace, rollback reserve, and replicas all count.

9.5 The 297-cell confirmatory design

The proposed logical matrix is:

```
6 deployable methods
× 3 reuse regimes {1,4,16}
× 3 active-capacity ratios {1,1/4,1/16}
× 3 paired stream-seed families
= 162

oracle on 27 operating cells
= 27

2 schedules {inline, deferred}
× 2 operators {identity, semantic consolidation}
× 3 reuse regimes
× 3 capacity ratios
× 3 seed families
= 108

total = 297 logical lifetime cells
```

The number of independent users per cell must come from blinded paired power simulation before final preregistration. No result is implied by this design.

9.6 Endpoints

The primary proposed endpoint is equal-weight lifetime utility AUC over cycles and three frozen probe blocks. Main scientific decisions ask:

- whether workload heterogeneity has material destination value;
- whether the past-only router beats a calibration-frozen benefit or efficiency comparator;
- whether reuse and memory pressure produce a destination-order reversal;
- whether semantic consolidation helps and whether deferred scheduling reduces lifecycle cost without a material schedule interaction.

The benchmark also measures protected retention, fixed-budget acquisition, exact/semantic/temporal/relational/procedural performance, internalization completeness, false memory, writer/retriever/reader error, router error, active and total state, sleep backlog, wake p99, movement, energy, deletion, and rollback.

9.7 Failure is an outcome

A method timeout, OOM, compiler exception, deadline miss, cap violation, invalid artifact, or method-produced missing probe is an observed resource failure. The stream remains in the intent-to-treat analysis, consumed work is charged, and affected utility is scored under the frozen failure rule. Complete-case analysis is sensitivity only.

Only shared protocol corruption that prevents all paired methods from being scored may invalidate a block. This prevents fragile methods from appearing strong by disappearing under pressure.

9.8 Statistical and reproducibility boundaries

The proposed target is a fixed equal-weight finite mixture of named workload and seed families, not an undefined superpopulation. Pairing preserves the identical stream, query order, and resource tuple across methods. Multiplicity, equivalence margins, failure-rate vetoes, model-selection rules, and missingness handling must be frozen before final evaluation.

Every claim should trace:

- source/version
- hypothesis
- config/data/code/environment
- raw checksum
- derived result
- claim
- figure/table
- manuscript sentence

The benchmark remains scientifically useful if the router fails, provided it yields deterministic streams, matched cross-substrate arms, complete accounting, separated retention and acquisition, a well-estimated knee/crossover or informative null, and reproducible provenance.

10. Open questions and research hypotheses

10.1 Is delayed execution itself beneficial?

[O/P] Existing work proves that delayed transformations can work, not that delay is causally superior. Compare immediate and snapshot-delayed updates with identical evidence, operator steps, destination, and future queries. Reject a sleep-specific benefit if delay provides no utility, safety, amortization, or wake-latency advantage after queue and publication cost.

10.2 What should be admitted?

[H] A past-only priority combining expected future gain, valid reuse, redundancy, surprise, volatility, privacy, and recovery value should outperform any single heuristic under heterogeneous streams. It fails if uniform, recency, or simple recurrence matches it at equal admitted bytes and compute across rare and correction slices.

10.3 Which destination should receive an item?

[H] The optimum is a hysteretic promotion/demotion policy, not one medium. Stable, high-reuse procedures should be promoted more aggressively; exact, volatile, uncertain, personal, or legally deletable evidence should remain reversible. If a single destination dominates the preregistered surface, the routing thesis narrows accordingly.

10.4 When should sleep run?

[H] Pressure-triggered sleep will beat a fixed clock only where novelty, interference, staleness, free capacity, queue load, or governance urgency varies enough to justify controller overhead. The experimental comparison must include one clock, nested clocks, fixed event counts, capacity pressure, recurrence, surprise, and a learned trigger.

10.5 Are NREM and REM useful engineering distinctions?

[H] Conservative replay/verification and exploratory recombination may need separate stages when their gradients or validation requirements conflict. Compare replay only, dream only, replay→dream, dream→replay, joint interleaving, and matched extra-data controls. If the ordered stages never beat a compute-matched joint or single-stage objective, the distinction is unnecessary in that regime.

10.6 Can sleep safely change the memory harness?

[H] Under high task heterogeneity, a frozen governance kernel plus task-specific write/read modules may dominate both a universal harness and full RHO-style harness rewrites after code generation, migration, sandboxing, validation, version count, and rollback are charged. A monotone winner across the full heterogeneity range rejects the intermediate regime.

10.7 Is visible memory the complete state?

[H] The gap between an explicit artifact and its effective execution state will grow with compression pressure and stateful reuse depth. Test text-identical restart, serialization, model migration, quantization, and opaque-secret deletion across layers and hops. Equivalence across all tests rejects the gap for the evaluated artifact class.

10.8 Does externalization free usable model capacity?

[H] Offloading facts improves new non-factual acquisition only if freed optimization or representational capacity is reachable by the new objective. At matched parameters, gradient tokens, and call budgets, lower factual leakage without improved acquisition rejects the simple “offload frees reasoning” mechanism.

10.9 What is the right deletion unit for embodied memory?

[E] [MEMORA](#) extends external sleep to video and action histories. [I] Participant, video, time span, entity, habit, workflow, preference, and parametric descendants can share one evidence source; this makes the dependency graph, rather than only text records, the relevant deletion scope. [O] Production consent and physical-world correction remain unresolved.

10.10 Which state is fixed, user-local, cohort, or shared?

[H] Promotion scope and medium should be selected jointly. User-local adapters may dominate shared weights even at lower raw compression because they bound negative transfer and rollback. The comparison must charge long-tail adapter storage, HBM residency, routing, and cold starts.

11. Governance, safety, and epistemic discipline

11.1 Promotion is a safety decision

A sleep learner can amplify poison, summarize away a constraint, merge distinct identities, install a malicious procedure, or publish a model delta whose source can no longer be deleted. A higher average score is insufficient.

[P] Candidate validation should include:

- current-task utility
- protected old-memory retention
- fixed-budget acquisition
- rare/failure/correction slices
- false memory and calibration
- poison and prompt-injection canaries
- authorization and privacy checks
- delete dependency completeness
- latency, bytes, compute, and energy
- rollback recovery

No aggregate gain should compensate for a preregistered protected-slice violation. Personal evidence should not be automatically promoted to shared weights.

11.2 Complete-state provenance

The serving artifact may include text, vector root, graph root, files, index, adapter, model version, authorization root, deletion epoch, and source range. These must be generation-bound. Updating text without its embedding produces key–payload drift. Publishing a vector without metadata produces a partial state. Clearing staging before the complete manifest is durable can lose evidence.

Source lineage should use opaque tenant-scoped handles rather than identifiers or reusable content fingerprints in operational manifests. Every generated summary, graph edge, adapter, skill, and harness should retain derivation edges or explicitly fail the governance gate.

11.3 Independent evaluation

When the same model generates dreams, repairs memory, judges quality, and ranks candidates, evidence is correlated. RHO and generated-memory systems make this concrete. Independent canaries, deterministic checks, model-family separation where feasible, and human review for high-risk promotion should be part of the cost rather than omitted.

11.4 Right to be forgotten versus right not to be served

Immediate denial and eventual physical removal are separate service objectives. A system may block retrieval quickly while asynchronously rebuilding derived state. It must report both. “No longer returned by top- k ” proves neither erasure nor absence of behavioral influence.

11.5 Claim-down rules

This field is especially vulnerable to metaphor and demo-driven overclaim. A rigorous program should enforce:

- finite results remain finite-regime claims;
 - absent biological evidence remains an algorithmic analogy;
 - simulations do not establish hardware latency or energy;
 - product documentation establishes behavior, not comparative effectiveness;
 - active-memory reduction is not total-state reduction;
 - significant but sub-material effects are not material support;
 - an equivalence interval overlapping its margin is inconclusive;
 - failed held-out prediction yields a regime-specific curve, not a renamed universal law;
 - no confirmatory arm is added after seeing final outcomes.
-

12. Synthesis: a promotion-and-demotion memory economy

The literature points toward neither all-external nor all-parametric memory. It supports a ladder:

```
raw event / exact transcript
    ↓ admission, deduplication
episodic text, vector, temporal graph
    ↓ merge, abstraction, verification
semantic or procedural external memory
    ↓ repeated-use compilation
latent/KV cartridge, user adapter, fast expert
    ↓ broad validation and high stable reuse
shared slow parametric memory
```

The arrows must also reverse. A volatile fact may be demoted from an adapter to external evidence. A saturated expert may be retired and rebuilt from canonical sources. A stale semantic view may be invalidated. An uncertain item may remain exact. A privacy-sensitive item may be denied and deleted. A low-value item may be deliberately forgotten.

This yields five principles.

1. External evidence is the default system of record

[I] Derived text, graphs, latents, adapters, and weights should be treated as materialized views while reversible evidence is legally and operationally retainable. Parametric memory is a compiled cache, not an unquestioned archive.

2. Test-time and sleep-time learning are complementary

Wake handles immediate adaptation and low-latency capture. Sleep handles expensive interleaving, verification, abstraction, dream generation, and promotion. The benchmark must include a strong wake learner so that every later gain is not falsely attributed to sleep.

3. Model memory and external memory share a control plane

Adapters still require catalogs, routers, compatibility checks, caches, source lineage, and revocation. External stores may require learned compilers and parametric readers. The systems boundary is not “model versus database”; it is an integrated state machine.

4. Capacity is governed by the first binding pressure

Storage, retrieval competition, slot availability, plasticity, queue load, HBM residency, deletion debt, or validation throughput may bind first. Monitoring old recall alone is insufficient.

5. Scaling laws should predict a boundary, not decorate a curve

The useful result is a held-out crossover or capacity knee that survives total-state accounting and claim-down rules. A universal exponent is neither necessary nor presently justified.

13. Limitations

This review is bounded to a local audited corpus with a 25 July 2026 cutoff. It does not prove that no additional public, proprietary, unpublished, patent, database, information-retrieval, or ML-systems precedent exists. Non-discovery is not novelty evidence.

Evidence quality is heterogeneous. It includes peer-reviewed neuroscience and continual-learning work, arXiv/OpenReview manuscripts, product documentation, tagged implementations, and a small number of low-evidence research notes. Numerical results are not pooled because models, datasets, phase boundaries, budgets, and scoring differ. Vendor-run comparisons are retained only with their source boundary.

Biological transfer is deliberately narrow. Replay and multiple timescales motivate algorithms, but the review does not adjudicate competing neuroscience theories or map brain structures onto software services.

The architecture is a proposal, not a production reference implementation. Linearizable serving heads, content-addressed manifests, canaries, and rollback reduce classes of failure but do not themselves establish privacy, security, or reliability.

The benchmark is preregistration-oriented and unexecuted. Its 297 logical cells, statistical family, resource envelope, and failure semantics are research design, not evidence for the hybrid router. Hardware claims beyond the named measured environment would require new prospective runs.

Finally, the position that canonical evidence should usually remain external is conditional on retention rights and security. Some evidence must be deleted immediately or cannot be stored at all. In those cases the system must state which recovery, correction, and audit guarantees it gives up.

14. Conclusion

Sleep-time compute is best understood not as a model imitating a sleeping brain, but as a **governed asynchronous memory lifecycle**. It begins with wake-derived evidence, runs a transformation outside the current action's critical path, and publishes persistent state for later use. Its scientific variables are the evidence admitted, corpus constructed, operator trained, destination chosen, validation passed, and resources consumed.

The historical evidence already establishes replay, pseudorehearsal, generated dreams, multiple timescales, explicit wake-sleep training, learned offline action selection, external memory rewriting, latent compilation, sparse parametric memory, and fast-to-slow expert transfer. The remaining target is therefore not “first sleep.” It is a cross-substrate policy that decides when and where experience should move while preserving total-state accounting, future utility, plasticity, provenance, correction, deletion, and rollback.

The central empirical bet is deliberately falsifiable: a past-only hybrid router may occupy a better lifetime frontier than tuned single-medium or static-mixture systems, and reuse, volatility, pressure, and governance may predict destination crossovers. If it does not, a well-powered null will still clarify which substrate or schedule is sufficient. If it does, the result must remain bounded to the measured stream, state ledger, and hardware regime.

The future learning stack may indeed separate pre-deployment training, wake/test-time adaptation, and sleep-time consolidation. Its infrastructure may likewise separate latency-critical wake execution, throughput-oriented sleep jobs, and a long-term memory fabric. But the decisive component is the control plane between them: immutable snapshots, explicit destination routing, independent validation, atomic publication, reversible versions, and measured capacity pressure. That is what turns a compelling metaphor into a scientific and systems research program.

References

The list below includes primary sources most directly used in the synthesis. Venue labels and lifecycle interpretations follow the bounded audit described above.

1. Crick, F., and Mitchison, G. [The Function of Dream Sleep](#). *Nature*, 1983.
2. Buzsáki, G. [A Two-Stage Model of Memory Trace Formation](#). *Neuroscience*, 1989.
3. McCloskey, M., and Cohen, N. J. [Catastrophic Interference in Connectionist Networks](#), 1989.
4. Ratcliff, R. [Connectionist Models of Recognition Memory](#). *Psychological Review*, 1990.
5. Wilson, M. A., and McNaughton, B. L. [Reactivation of Hippocampal Ensemble Memories During Sleep](#). *Science*, 1994.
6. McClelland, J. L., McNaughton, B. L., and O'Reilly, R. C. [Why There Are Complementary Learning Systems in the Hippocampus and Neocortex](#), 1995.
7. Hinton, G. E., et al. [The “Wake-Sleep” Algorithm for Unsupervised Neural Networks](#). *Science*, 1995.
8. Robins, A. [Catastrophic Forgetting, Rehearsal and Pseudorehearsal](#). *Connection Science*, 1995.
9. Skaggs, W. E., and McNaughton, B. L. [Replay of Neuronal Firing Sequences in Rat Hippocampus During Sleep](#). *Science*, 1996.
10. Louie, K., and Wilson, M. A. [Temporally Structured Replay of Awake Hippocampal Ensemble Activity During Rapid Eye Movement Sleep](#). *Neuron*, 2001.
11. Tononi, G., and Cirelli, C. [Sleep and Synaptic Homeostasis](#). *Brain Research Bulletin*, 2003.
12. Fusi, S., Drew, P. J., and Abbott, L. F. [Cascade Models of Synaptically Stored Memories](#). *Neuron*, 2005.
13. Fusi, S., and Abbott, L. F. [Limits on the Memory Storage Capacity of Bounded Synapses](#). *Nature Neuroscience*, 2007.
14. Lahiri, S., and Ganguli, S. [A Memory Frontier for Complex Synapses](#). NeurIPS, 2013.
15. Tononi, G., and Cirelli, C. [Sleep and the Price of Plasticity](#). *Neuron*, 2014.
16. Benna, M. K., and Fusi, S. [Computational Principles of Synaptic Memory Consolidation](#). *Nature Neuroscience*, 2016.
17. Kumaran, D., Hassabis, D., and McClelland, J. L. [What Learning Systems Do Intelligent Agents Need?](#). *Trends in Cognitive Sciences*, 2016.
18. Rusu, A. A., et al. [Progressive Neural Networks](#), 2016.
19. Kirkpatrick, J., et al. [Overcoming Catastrophic Forgetting in Neural Networks](#). *PNAS*, 2017.
20. Shin, H., et al. [Continual Learning with Deep Generative Replay](#). NeurIPS, 2017.
21. Lopez-Paz, D., and Ranzato, M. [Gradient Episodic Memory for Continual Learning](#). NeurIPS, 2017.
22. Kamra, N., et al. [Deep Generative Dual Memory Network for Continual Learning](#), 2017.
23. Aljundi, R., et al. [Memory Aware Synapses](#). ECCV, 2018.

24. Kemker, R., and Kanan, C. [FearNet: Brain-Inspired Model for Incremental Learning](#). ICLR, 2018.
25. Schwarz, J., et al. [Progress & Compress](#). ICML, 2018.
26. Fachechi, A., et al. [Dreaming Neural Networks](#). *Neural Networks*, 2019.
27. Lample, G., et al. [Large Memory Layers with Product Keys](#). NeurIPS, 2019.
28. Sun, Y., et al. [Test-Time Training with Self-Supervision](#). ICML, 2020.
29. González, O. C., et al. [Can Sleep Protect Memories from Catastrophic Forgetting?](#). *eLife*, 2020.
30. van de Ven, G. M., et al. [Brain-Inspired Replay for Continual Learning](#). *Nature Communications*, 2020.
31. Deperrois, N., et al. [Learning Cortical Representations Through Perturbed and Adversarial Dreaming](#). *eLife*, 2022.
32. Singh, D., Norman, K. A., and Schapiro, A. C. [A Model of Autonomous Interactions Between Hippocampus and Neocortex Driving Sleep-Dependent Memory Consolidation](#). *PNAS*, 2022.
33. Golden, R., et al. [Sleep Prevents Catastrophic Forgetting in Spiking Neural Networks by Forming a Joint Synaptic Weight Representation](#). *PLOS Computational Biology*, 2022.
34. Tadros, T., et al. [Sleep-Like Unsupervised Replay Reduces Catastrophic Forgetting in Artificial Neural Networks](#). *Nature Communications*, 2022.
35. Harun, Y., et al. [SIESTA: Efficient Online Continual Learning with Sleep](#). TMLR, 2023.
36. Shinn, N., et al. [Reflexion: Language Agents with Verbal Reinforcement Learning](#), 2023.
37. Park, J. S., et al. [Generative Agents](#), 2023.
38. Wang, W., et al. [LongMem](#). NeurIPS, 2023.
39. Packer, C., et al. [MemGPT: Towards LLMs as Operating Systems](#), 2023.
40. Sorrenti, S., et al. [Wake-Sleep Consolidated Learning](#). *IEEE TNNLS*, 2025.
41. Sun, Y., et al. [Learning to \(Learn at Test Time\)](#), 2024.
42. Taylor, C., et al. [Patch-Based Contrastive Learning and Memory Consolidation for Online Unsupervised Continual Learning](#). PMLR, 2025.
43. Allen-Zhu, Z., and Li, Y. [Physics of Language Models: Knowledge Capacity Scaling Laws](#), 2024.
44. Dohare, S., et al. [Loss of Plasticity in Deep Continual Learning](#). *Nature*, 2024.
45. Berges, V.-P., et al. [Memory Layers at Scale](#). ICML, 2025.
46. Behrouz, A., et al. [Titans: Learning to Memorize at Test Time](#). NeurIPS, 2025.
47. Rasmussen, L. M., et al. [Zep: A Temporal Knowledge Graph Architecture for Agent Memory](#), 2025.
48. Lin, K., et al. [Sleep-time Compute](#), 2025.
49. [Mem0](#). ECAI, 2025.
50. Morris, J. X., et al. [How Much Do Language Models Memorize?](#), 2025.
51. Tutuncuoglu, B. T. [Dream-Augmented Neural Networks](#). SSRN, 2025.
52. Spens, E., Burgess, N., and Behrens, T. [Modelling the Control of Offline Processing with Reinforcement Learning](#). NeurIPS, 2025.
53. Lin, Z., et al. [Continual Learning via Sparse Memory Finetuning](#), 2025.
54. [Cartridges](#), 2025.
55. Behrouz, A., et al. [Language Models Need Sleep](#), 2026.

56. Kerestecioglu, E., et al. [Human-Inspired Memory Architecture for LLM Agents](#), 2026.
57. Pan, W., et al. [Retrospective Harness Optimization](#), 2026.
58. Ye et al. [Auto-Dreamer](#), 2026.
59. Kontonis, V., et al. [MEMENTO](#), 2026.
60. Xia, et al. [MEMORA](#), 2026.
61. Yang, et al. [PlugMem](#), 2026.
62. Lin, et al. [MemMA](#), 2026.
63. Mouchon, L. [Surprise as a Signal for Plasticity and Metacognition](#), 2026.
64. O'Neill, C. [Can a Language Model Learn Facts Continually in Its Weights?](#), 2026.
65. Colaco, A. G., and Lahjouji, N. [What to Keep, What to Forget: A Rate-Distortion View of Memory Compaction in LLMs and Agents](#), 2026.
66. [Understanding LoRA as Knowledge Memory: An Empirical Analysis](#), 2026.
67. [TiMem](#). Findings of ACL, 2026.
68. [RecMem: Recurrence-Based Memory Consolidation](#). Findings of ACL, 2026.
69. [GAM: Hierarchical Graph-Based Agentic Memory](#). ACL, 2026.
70. [Are We Ready For An Agent-Native Memory System?](#), 2026.
71. [Do Language Models Need Sleep? Offline Recurrence for Improved Online Inference](#), 2026.
72. [Cartridges at Scale](#), 2026.
73. [When Continual Learning Moves to Memory](#), 2026.
74. [Useful Memories Become Faulty When Continuously Updated by LLMs](#), 2026.
75. [Episodic-to-Semantic Consolidation Without Identity Drift](#), 2026.
76. [MemDefrag: Latent Memory Defragmentation for Large Language Models](#), 2026.
77. [MEMORA: Embodied Action Memory from Egocentric Videos for Reasoning and Planning](#), 2026.
78. Golden, R., et al. [BioRxiv precursor to the 2022 PLOS sleep study](#), 2019.
79. Zenke, F., Poole, B., and Ganguli, S. [Continual Learning Through Synaptic Intelligence](#). ICML, 2017.
80. Mallya, A., and Lazebnik, S. [PackNet](#). CVPR, 2018.
81. Serrà, J., et al. [Hard Attention to the Task](#). ICML, 2018.
82. Chaudhry, A., et al. [On Tiny Episodic Memories in Continual Learning](#), 2019.
83. Liang et al. [Learning Personalized Agents from Human Feedback](#), 2026.